

Arquitectura y decisiones técnicas

Mosaico — interfaz generativa para BANORTECH

Arquitectura de la solución

Reto UI Generativa · Banorte × Tec de Monterrey

13 de septiembre de 2026

1 Arquitectura del sistema

La solución se organiza en tres procesos: el cliente móvil, el servidor de orquestación y el servidor de datos MCP. El modelo de lenguaje ocupa el centro de la experiencia; el resto de los componentes ejecutan, pero no deciden.

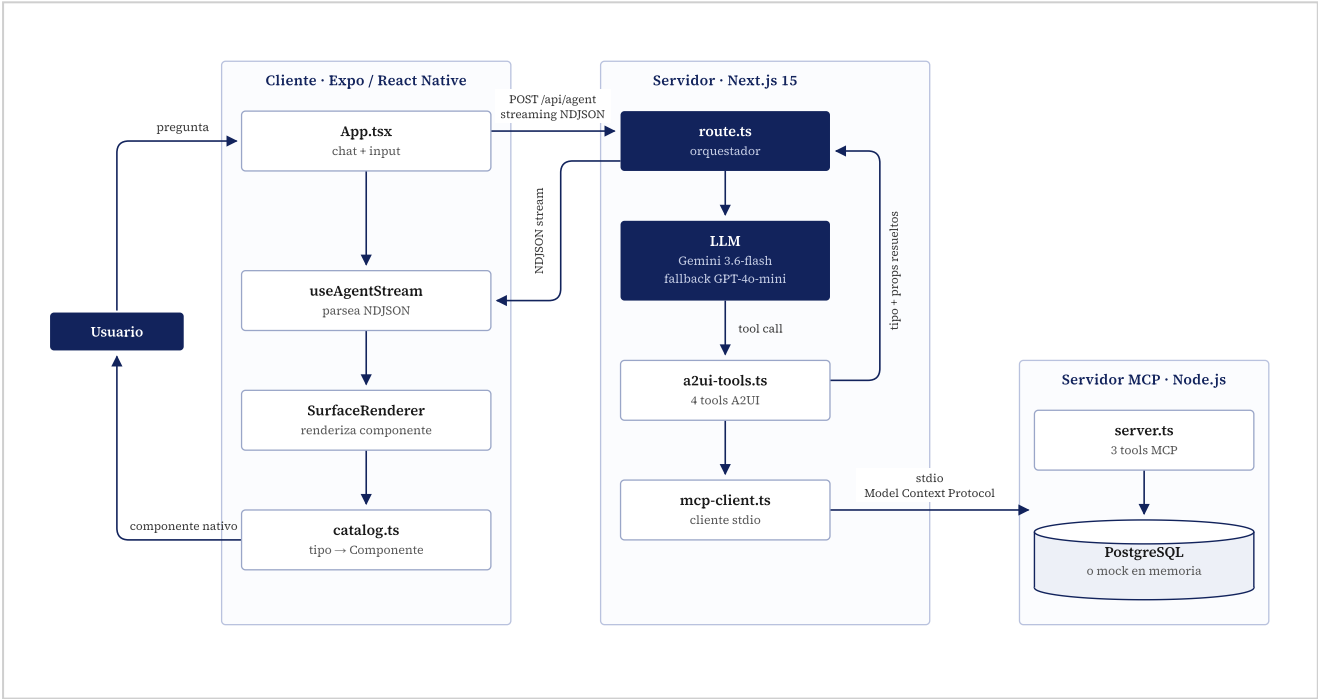


Figura 1. Arquitectura de referencia. El ciclo se cierra en el usuario: cada interacción con la interfaz generada regresa al orquestador como contexto.

2 Flujo de un mensaje

Secuencia completa desde que el usuario formula una pregunta hasta que la interfaz correspondiente se renderiza en el dispositivo.

- El usuario escribe una pregunta en `App.tsx`.
- `useAgentStream` ejecuta `POST /api/agent` y lee la respuesta en streaming, línea a línea.
- `route.ts` invoca `streamText` con el modelo y las cuatro herramientas A2UI.

- 4El modelo determina qué herramienta invocar (por ejemplo, `mostrarProgresoMeta`).
- 5La herramienta consulta el servidor MCP y obtiene datos reales de PostgreSQL, o del mock en memoria si no existe `DATABASE_URL` .
- 6El servidor emite una línea NDJSON: `{"type":"surface","tipo":"RastreadorMetas","props":{...}}` .
- 7`SurfaceRenderer` localiza `"RastreadorMetas"` en `catalog.ts` y renderiza el componente nativo con las props ya resueltas.

3 Decisiones técnicas

3.1 Selección de modelo

Dimensión	Gemini 3.6 Flash (primario)	GPT-4o Mini (respaldo)
Costo	Free tier (cuota baja por key)	Pay-per-use
Latencia	~1~2 s en streaming	~1.5 s en streaming
Tool calling	Soportado	Soportado
Estabilidad de IDs	Los IDs de modelo cambian; se usan explícitos, no alias	Más estable
Rol	Primario	Fallback automático si Gemini falla antes de emitir contenido

Justificación. Gemini cubre el free tier del hackathon; ante una falla, OpenAI asume la respuesta sin reiniciar el stream. Se descartó Claude Sonnet porque requiere una key de pago y no aporta una ventaja diferencial para el dominio financiero dentro de este alcance.

Regla operativa. No se emplean alias como `gemini-flash-latest` , ya que el modelo al que resuelven puede cambiar y quedar sujeto a una cuota de 20 solicitudes diarias. Se utilizan siempre identificadores explícitos, como `gemini-3.6-flash` .

3.2 Protocolo de interfaz

Dimensión	A2UI-lite · NDJSON (elegido)	RSC / streamUI	REST JSON puro
Formato	JSON declarativo <code>{tipo, props}</code>	JSX serializado (código)	JSON en una sola respuesta
Clientes soportados	Cualquiera: RN, web, Flutter	Solo React	Cualquiera
Streaming	Sí, línea a línea	Sí	No
Riesgo de alucinación	Bajo; el modelo elige qué mostrar y los datos provienen del MCP	Medio	Alto; el modelo genera cifras como texto
Complejidad	Baja; NDJSON es texto plano	Alta; requiere runtime RSC	Baja

Justificación. RSC fue el enfoque inicial y se descartó porque `streamUI` transmite JSX —código React ya renderizado—, un formato que React Native no puede interpretar. A2UI-lite transmite `{tipo, props}` puro: el cliente recibe datos y los renderiza con su propio catálogo de componentes nativos. Ningún JSX ni código ejecutable cruza la red.

Alcance del subconjunto. El protocolo transmite cada bloque ya resuelto en una sola emisión, en lugar del streaming granular componente por componente del A2UI completo oficial. Es suficiente para el alcance del reto y facilita la depuración.

3.3 Infraestructura

Componente	Elegido	Alternativas consideradas	Razón
Cliente	Expo v57 / React Native	Next.js web, Flutter	Un solo codebase para web, Android e iOS; Expo Go para demostración rápida
Servidor	Next.js 15 (App Router)	Express, Fastify, Hono	Route handlers listos para serverless; streaming HTTP nativo; Vercel AI SDK integrado
Transporte MCP	stdio (proceso hijo)	HTTP/SSE	Sin puerto adicional en desarrollo; migrable a HTTP si se despliegan por separado
Base de datos	PostgreSQL (vía MCP)	MySQL, MongoDB, SQLite	Soportada por el MCP SDK; esquema relacional natural para cuentas y transacciones
Historial de chat	SQLite local (cliente)	Postgres, sin persistencia	Dato personal y volátil; no requiere sincronización
Modo desarrollo	Mock en memoria (sin <code>DATABASE_URL</code>)	Base de datos siempre activa	Permite desarrollar y demostrar sin PostgreSQL en ejecución

Persistencia: receta en lugar de resultado. El tablero no almacena los valores calculados (`porcentaje: 62`), sino la receta que los genera (`{tool: "mostrarProgresoMeta", parametros: {metaId: "meta-1"}}`). Al rehidratar, el servidor ejecuta la herramienta y recupera el dato actualizado, evitando cifras desactualizadas.